



WEB APPLICATION BASICS

HTTP Protocol & Security Fundamentals Reference Sheet



SECURITY GUIDE

1 HTTP REQUEST STRUCTURE

An HTTP Request consists of a **Request Line**, **Headers**, and an optional **Body**.

THE REQUEST LINE FORMAT

METHOD /path/to/resource HTTP/version

- Example: POST /login HTTP/1.1

2 HTTP METHODS & SECURITY CONSIDERATIONS

METHOD	DESCRIPTION	SECURITY FOCUS
GET	Fetches data from the server without modifying state.	Never send sensitive data (tokens, passwords) in parameters; URL parameters appear in plaintext logs/browser history.
POST	Submits data to create resources or submit forms.	Always validate and sanitize inputs to prevent SQL Injection, XSS, and parameter tampering.
PUT	Replaces or updates an entire target resource.	Enforce strict authorization controls to prevent unauthorized record overwrites.
DELETE	Removes a specified resource from the server.	Enforce strict authorization controls to avoid unauthorized deletion of records.
PATCH	Applies partial modifications to a resource.	Validate schema updates strictly to avoid unexpected or partial data corruption.
HEAD	Same as GET, but returns headers only (no response body).	Used to inspect resource metadata efficiently without downloading content.
OPTIONS	Queries the server for allowed HTTP methods on a resource.	Disable if unused to limit attacker recon opportunities.
TRACE	Echoes back the received request for debugging.	Disable on production servers to prevent Cross-Site Tracing (XST) attacks.
CONNECT	Establishes a tunnel (typically used for HTTPS proxying).	Essential for encrypted channel handshakes.

3 HTTP REQUEST HEADERS

Request headers send additional client metadata to the web server:

HEADER	EXAMPLE	DESCRIPTION
Host	Host: tryhackme.com	Specifies the domain name/server targeted by the client.
User-Agent Referer	User-Agent: Mozilla/5.0 Referer: https://google.com	Identifies client software/browser making request. Identifies preceding URL that linked to current page.
Cookie	Cookie: session=123abc; role=admin	Sends stored session IDs or state stored by the browser.
Content-Type	Content-Type: application/json	Declares the formatting/MIME type of payload in request body.
Content-Length	Content-Length: 43	Specifies the exact size of the request body in bytes.

4 REQUEST BODY FORMATS

FORMAT	HEADER VALUE	STRUCTURE EXAMPLE	USE CASE
URL-Encoded	application/x-www-form-urlencoded	username=user1&pass=123	Standard HTML form submissions.
Form Data	multipart/form-data; boundary=---Boundary	Uses boundary strings separating fields/binary streams.	File uploads and mixed binary/text inputs.
JSON	application/json	{"name": "Alice", "age": 25}	Modern APIs and modern web applications.
XML	application/xml	<user> <name>Alice</name> </user>	Legacy systems and SOAP APIs.

5 HTTP RESPONSES & STATUS CODES

STATUS LINE FORMAT

HTTP/version Status_Code Reason_Phrase

- Example: HTTP/1.1 200 OK

STATUS CODE CATEGORIES

CODE RANGE	CATEGORY	DEFINITION	COMMON EXAMPLE
100–199	Informational	Request received; continuing process.	100 Continue
200–299	Success	Request was successfully received and executed.	200 OK
300–399	Redirection	Further action/redirect needed to reach resource.	301 Moved Permanently
400–499	Client Error	Bad request syntax or missing	404 Not Found

6 COMMON HTTP RESPONSE HEADERS

HEADER	EXAMPLE	MEANING & SECURITY NOTES
Date	Date: Fri, 23 Aug 2024 10:43:21 GMT	Server timestamp when the response was constructed.
Content-Type	Content-Type: text/html; charset=utf-8	MIME type and character encoding of response body data.
Server	Server: nginx	Server software version. Best Practice: Obscure or strip to prevent banner grabbing.
Set-Cookie	Set-Cookie: ID=123; Secure; HttpOnly	Instructs client to store cookies. Best Practice: Enforce HttpOnly and Secure flags.
Cache-Control	Cache-Control: max-age=600, no-cache	Rules for browser/proxy caching behavior.
Location	Location: /dashboard	Target URL for 3xx redirects. Best Practice: Validate to prevent Open Redirects.

7 HTTP SECURITY HEADERS

1. Content-Security-Policy (CSP)

Mitigates Cross-Site Scripting (XSS) and data injection by declaring whitelisted sources.

```
Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.example.com; style-src 'self'
```

- `'self'`: Restricts sources strictly to the same origin (domain, port, protocol).
- `script-src`: Restricts allowed origins for JavaScript execution.
- `style-src`: Restricts allowed origins for CSS stylesheets.

2. Strict-Transport-Security (HSTS)

Forces browsers to establish connections over encrypted HTTPS connections only.

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

- `max-age`: Time in seconds for enforcing the HTTPS rule.
- `includeSubDomains`: Applies HSTS policy to all subdomains.
- `preload`: Authorizes browser vendors to hardcode the domain into HSTS lists.

3. X-Content-Type-Options

Prevents MIME-sniffing vulnerabilities.

```
X-Content-Type-Options: nosniff
```

nosniff: Forces browser to strictly honor declared Content-Type.

4. Referrer-Policy

Controls path/domain leakage upon navigation.

```
Referrer-Policy: strict-origin-when-cross-origin
```

- no-referrer**: Drops Referer header completely.
- same-origin**: Sends referrer data only on same domain.
- strict-origin**: Sends domain origin, maintaining HTTPS checks.